Simulador de control PID y control PID real de velocidad para un motor de corriente continua

Carlos Andrés Batista Figueroa — T00078355
Programa de Ingeniería Mecatrónica, Facultad de Ingeniería, Universidad Tecnológica de Bolívar
Asignatura: Control Inteligente — Docente: Ing. Ph.D. Efraín Rodríguez
Cartagena de Indias D. T. y C., Colombia — 14 de septiembre de 2026

***Abstract*—This report presents the design, implementation and experimental validation of an integrated platform for the study of PID speed control of a direct current motor. The platform has two complementary parts. The first one is a desktop simulator written in Python that solves the electrical and mechanical equations of an armature-controlled DC motor using a fourth order Runge–Kutta integrator, and displays in real time the closed loop block diagram, the tracking error, the controller output and the response of the plant. The second one is a physical control loop built with an Arduino Uno, an L298N H-bridge and an FC-03 optical slot sensor. Its firmware measures speed edge to edge, filters spurious pulses and applies a discrete PID with feedforward, conditional integration anti-windup and derivative action on the measurement. The same desktop application connects to the board through the serial port, so that the setpoint and the three gains can be modified while the motor is running. The experimental tests showed a stable step response without steady state error between approximately 25 and 150 rpm, and a progressive degradation of the measurement above that range, caused by the limited resolution of the 20-slot encoder.**

***Index Terms*—PID control, DC motor, PWM, optical encoder, Arduino, real time simulation.**

I. Introducción

El control automático de velocidad es uno de los problemas clásicos de la ingeniería mecatrónica y está presente en casi cualquier máquina que deba moverse de forma predecible. Dentro de ese campo, los motores de corriente continua son el actuador más común en aplicaciones de baja potencia, porque su velocidad depende de forma casi lineal del voltaje aplicado y porque existen etapas de potencia económicas para gobernarlos. Sin embargo, un motor sometido únicamente a un voltaje fijo cambia su velocidad ante cualquier variación de la carga o de la alimentación, lo que obliga a medir la salida y corregirla de manera continua. Esa idea, la realimentación, es el eje de la teoría de control, y su implementación más difundida es el controlador proporcional–integral–derivativo. El presente informe se sitúa en ese contexto y documenta un trabajo que combina herramientas de simulación con hardware electrónico de bajo costo.

II. Objetivos

II-A. Objetivo general

Diseñar, implementar y validar experimentalmente una plataforma de control PID de velocidad para un motor de corriente continua, que integre en una misma aplicación la simulación del lazo cerrado y el gobierno de un motor físico a través de un microcontrolador.

II-B. Objetivos específicos

1. Obtener el modelo dinámico del motor a partir de la ley de voltajes de Kirchhoff y del equilibrio de pares del rotor, e implementarlo en un simulador de tiempo real con un PID de ganancias ajustables.
2. Implementar en un Arduino Uno un firmware capaz de medir la velocidad del eje con un sensor óptico y de regular el ciclo útil de la señal PWM aplicada al motor mediante un PID discreto.
3. Desarrollar una interfaz de escritorio que comunique el computador con el microcontrolador por el puerto serie, y evaluar experimentalmente la respuesta del lazo cerrado frente a cambios de consigna.

III. Marco teórico

Todo sistema de control nace de una pregunta sencilla: cómo lograr que una variable física siga el valor que el diseñador desea. Cuando el actuador se acciona sin verificar el resultado se habla de *lazo abierto*, un esquema simple pero incapaz de corregir perturbaciones. Al medir la salida y compararla con la consigna aparece el *lazo cerrado*, en el que la diferencia entre la referencia $r(t)$ y la medición $y(t)$ define el error $e(t) = r(t) - y(t)$, la señal sobre la cual actúa el controlador (Ogata, 2010). El controlador más extendido en la industria es el PID, que combina tres acciones sobre ese error:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt} \qquad (1)$$

La acción proporcional responde al error presente, la integral elimina el error en estado estacionario acumulando el error pasado, y la derivativa anticipa su tendencia y amortigua la respuesta (Åström y Hägglund, 1995). Dos precauciones son indispensables en la práctica: cuando el actuador se satura, el término integral sigue creciendo sin efecto útil (*windup*) y debe limitarse con un mecanismo de *anti-windup*; y derivar el error produce un salto brusco de la salida al cambiar la consigna, que se evita derivando sobre la medición (Åström y Hägglund, 2001).

La planta de este trabajo es un motor de corriente continua controlado por armadura, cuyo modelo une dos dominios. En el eléctrico se aplica la ley de voltajes de Kirchhoff a la malla

de armadura: las caídas en la resistencia, en la inductancia y en la fuerza contraelectromotriz igualan el voltaje aplicado,

$$V_a = R_a\, i_a + L_a \frac{di_a}{dt} + K_e\, \omega \tag{2}$$

donde $K_e\,\omega$ es la tensión que el propio giro induce y que se opone a la alimentación (Nise, 2011). En el mecánico se aplica la segunda ley de Newton para rotación, es decir, la sumatoria de pares sobre el eje igualada al producto de la inercia total por la aceleración angular:

$$J \frac{d\omega}{dt} = K_t\, i_a - B\,\omega - T_L \tag{3}$$

Aquí J es la suma de las inercias del rotor y de todo lo acoplado al eje (disco ranurado, engranajes y carga), B el coeficiente de fricción viscosa y T_L el par de carga. Ambas ecuaciones están acopladas: la corriente genera par y la velocidad genera contratensión. Su solución conjunta produce una respuesta de segundo orden con dos constantes de tiempo, la eléctrica $\tau_e = L_a/R_a$ y la mecánica $\tau_m = J/(B{+}K_tK_e/R_a)$; como normalmente $\tau_e \ll \tau_m$, el motor se comporta en la práctica como un sistema de primer orden (Dorf y Bishop, 2011).

Un microcontrolador no entrega voltaje analógico, de modo que el voltaje de armadura se sintetiza con modulación por ancho de pulso (PWM): se conmuta la alimentación a frecuencia constante y se varía la fracción de tiempo encendida, el ciclo útil D. La inercia del motor promedia esos pulsos, así que percibe un voltaje medio $\overline{V}_a = D{\cdot}V_{cc}$ y su velocidad se regula de forma casi lineal sin disipar potencia en resistencias (Fatma y Hamid, 2019). El sentido de giro lo impone un puente H, que invierte la polaridad. La realimentación proviene de un encoder óptico incremental: un disco de N ranuras interrumpe un haz infrarrojo, de modo que contar pulsos en un intervalo conocido equivale a medir velocidad. Por último, el PID se discretiza con un período de muestreo T_s que debe ser mucho menor que la constante de tiempo dominante de la planta (Ogata, 2010).

IV. Materiales y equipos

- **Motorreductor DC amarillo tipo TT** (1:48, 3 a 6 V): planta.
- **Arduino UNO R3** (ATmega328P a 16 MHz): unidad de control.
- **Módulo puente H L298N**: etapa de potencia; aplica el PWM y fija el sentido de giro.
- **Sensor óptico de herradura FC-03** (comparador LM393) y disco ranurado de 20 ranuras: transductor de velocidad.
- **Convertidor DC–DC reductor LM2596**: ajuste de la tensión de alimentación.
- **Dos baterías de litio 18650 de 3,7 V** con portapilas: fuente de potencia (unos 6,5 V en serie).
- **Complementos**: cable USB A–B, cables de conexión, base de madera y computador con Python 3.14 y VS Code con PlatformIO.

V. Metodología de desarrollo

V-A. *Enfoque y orden de trabajo*

El proyecto se desarrolló de forma incremental, verificando cada etapa antes de pasar a la siguiente. Primero se construyó el simulador del lazo cerrado, que permitió entender el comportamiento esperado sin arriesgar el hardware. Después se montó el circuito y se programó el PWM en lazo abierto. A continuación se incorporó el sensor óptico y, con la medición disponible, se cerró el lazo con un PID discreto. Por último, la aplicación de escritorio se amplió con una tercera ventana que se comunica con el Arduino por el puerto serie, de modo que la misma herramienta permite simular y gobernar el motor real.

V-B. *Modelado y simulación del lazo cerrado*

El simulador integra las ecuaciones del motor con Runge–Kutta de cuarto orden, manteniendo constante el voltaje de armadura durante cada período de muestreo. Como la constante de tiempo eléctrica es mucho menor que el período del controlador, cada período se subdivide en varios subpasos para conservar la precisión. El controlador es un PID en paralelo con anti-windup por integración condicional, derivada sobre la medición y filtro pasa-bajas en el término derivativo. La referencia puede ser escalón, onda cuadrada o senoidal (Figs. 1 y 2).



Fig. 1. Ventana *Control PID*: diagrama del lazo cerrado con el valor vivo de cada señal, error, salida del controlador y salida del proceso.

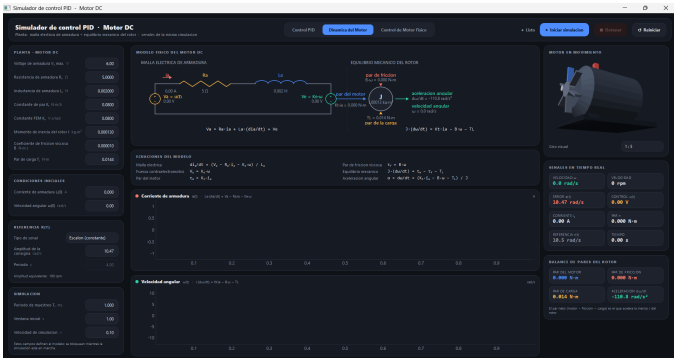


Fig. 2. Ventana *Dinámica del Motor*: malla de armadura, equilibrio mecánico del rotor, ecuaciones y balance de pares en tiempo real.

V-C. *Medición de la velocidad*

La velocidad no se mide contando pulsos en una ventana fija, sino midiendo el tiempo exacto entre flancos del sensor, lo que elimina el error de cuantización. Dentro de una rutina de interrupción se registran los pulsos válidos y el instante del último flanco; cada 25 ms se calcula

$$\text{RPM} = \frac{60 \times 10^6 \cdot N_{\text{pulsos}}}{\Delta t_{\mu s} \cdot N_{\text{ranuras}}} \tag{4}$$

donde $\Delta t_{\mu s}$ es el intervalo entre el último flanco de la ventana actual y el de la anterior, y $N_{\text{ranuras}} = 20$. El resultado pasa por una mediana de las cinco últimas medidas, que descarta los picos aislados generados por el ruido de conmutación del PWM.

V-D. *Control del motor físico*

El error se normaliza respecto al fondo de escala del motor ($\text{rpm}_{\text{máx}} = 600$), con lo que las ganancias quedan adimensionales y en rangos manejables. Además se añade una prealimentación que compensa la zona muerta del conjunto motor–puente H, medida en 15 % de PWM:

$$ff = \text{pwm}_{\text{mín}} + |SP| \cdot \frac{100 - \text{pwm}_{\text{mín}}}{\text{rpm}_{\text{máx}}} \tag{5}$$

$$u = \text{sat}_{[0,100]}\,(ff + P + I + D) \tag{6}$$

Así el PID solo corrige la diferencia entre el modelo y el comportamiento real, en lugar de construir toda la señal de control desde el integrador.

V-E. *Funciones de cada ventana*

La aplicación se organiza en tres vistas seleccionables desde la cabecera:

- **Control PID** (Fig. 1): diagrama de bloques del lazo con el valor vivo de cada señal; campos de planta, condiciones iniciales, referencia y simulación; gráficas de $e(t)$, de $u(t)$ con sus líneas de saturación y de $y(t)$; deslizadores de K_p, K_i y K_d y casillas de anti-windup y derivada sobre la medición.
- **Dinámica del Motor** (Fig. 2): esquema del modelo físico, ecuaciones, gráficas de corriente de armadura y de velocidad angular, y balance de pares del rotor.
- **Control de Motor Físico** (Figs. 3 a 11): tarjeta de conexión con selector de puerto y baudios; vista 3D del motor; gráfica de RPM contra tiempo con la consigna en escalones; medición del encoder (RPM, PWM y pulsos); campos de consigna y de ranuras por vuelta con botón de parada; estado del firmware; monitor del puerto serie con envío manual; y deslizadores de las tres ganancias.

VI. Firmware utilizado

VI-A. *Código A: firmware del Arduino Uno*

Es el programa que corre en el microcontrolador y constituye el controlador en tiempo real. Está escrito en C++ sobre el framework de Arduino y no usa esperas bloqueantes: el lazo principal solo atiende el puerto serie y revisa dos temporizadores basados en `millis()`. El primero se dispara

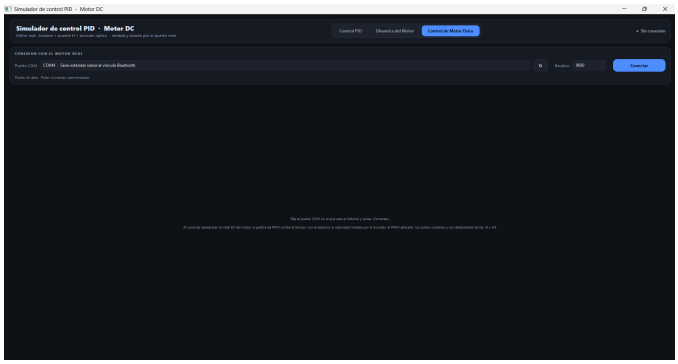


Fig. 3. Vista del motor físico antes de conectar: solo aparece la tarjeta de conexión con el selector de puerto y de baudios.

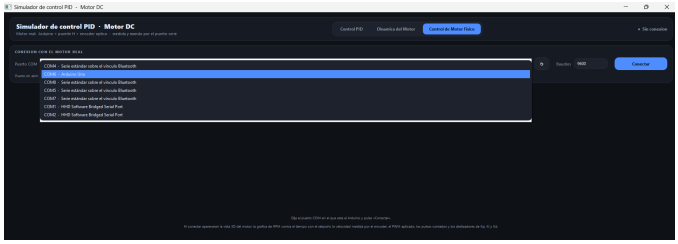


Fig. 4. Selección del puerto serie: la aplicación enumera los puertos del equipo e identifica el Arduino Uno en COM6.

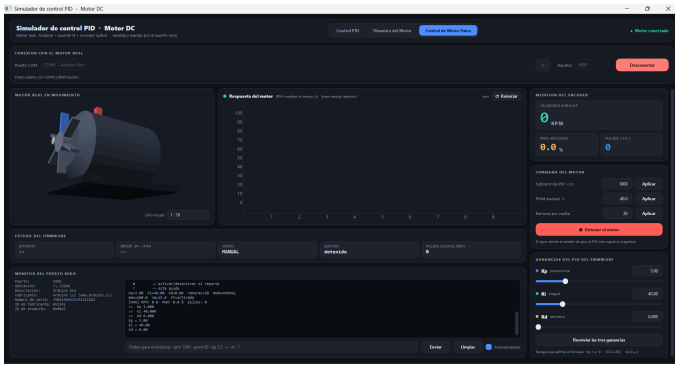


Fig. 5. Interfaz recién conectada en modo manual; el monitor serie muestra la ayuda del firmware con los valores por defecto.

cada 25 ms (40 Hz) y ejecuta la medición y el cálculo del PID; el segundo, cada 250 ms (4 Hz), imprime una línea de reporte con la consigna, las RPM, el PWM, el error y los pulsos. El conteo ocurre de forma asíncrona en una interrupción asociada al pin D2, que descarta los flancos que no permanecen en bajo al menos 8 microsegundos y los que llegan antes de 250 microsegundos del anterior. El controlador calcula la prealimentación, aplica las tres acciones sobre el error normalizado, limita el resultado entre 0 y 100 % y lo escribe en el puente H junto con el sentido de giro. Todos los parámetros se modifican en caliente con comandos de texto a 9600 baudios (Fig. 6).

VI-B. *Código B: aplicación de escritorio en Python*

Está escrita con PyQt5 y pyqtgraph y cumple dos funciones independientes. Como simulador, integra el modelo con Runge–Kutta de cuarto orden, calcula el PID, almacena el

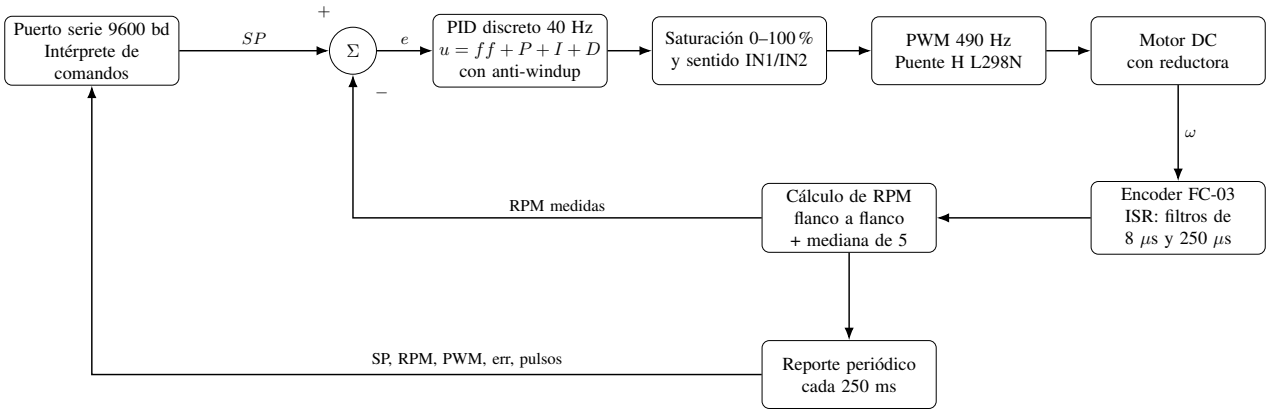


Fig. 6. Diagrama de bloques general del código A (firmware del Arduino Uno).

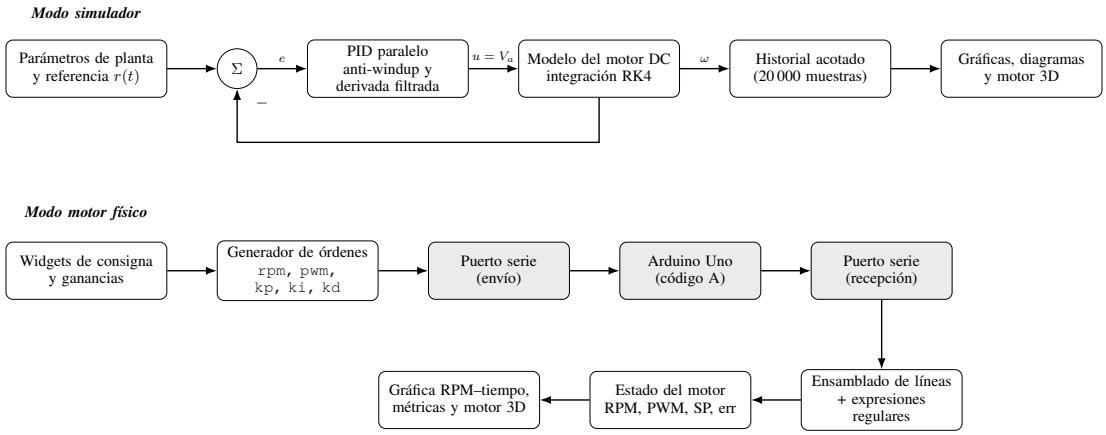


Fig. 7. Diagrama de bloques general del código B (aplicación de escritorio en Python).

ensayo en un historial de memoria acotada y dibuja en tiempo real el diagrama de bloques, las señales del lazo y un modelo tridimensional del motor renderizado por software. Como interfaz del motor real, abre el puerto con `QSerialPort`, acumula los bytes recibidos, los separa en líneas y extrae con expresiones regulares la velocidad, el PWM, los pulsos, la consigna, el error, el modo y el sentido de giro; con esos datos actualiza las métricas, anima el motor 3D y traza la respuesta RPM contra tiempo. En sentido contrario, los widgets generan las órdenes de texto que se envían al firmware, agrupando los cambios de los deslizadores para no saturar el canal. La aplicación es de un solo hilo y respeta un presupuesto de tiempo por fotograma para que la ventana nunca deje de responder (Fig. 7).

Enlace al proyecto completo: https://github.com/CarlosBatista08/SIMULATOR-CONTROLLER-PID-FOR-DC-MOTOR

VII. Pruebas y evidencias

A continuación se recogen las evidencias del montaje y de los ensayos realizados sobre el motor real.

Enlace al video de funcionamiento: https://tecnoutb-my.sharepoint.com/:v:/g/personal/cabatista_utb_edu_co/IQDiiL4pH_9JRo1WGevNPPJoAc_cnvtuTesJkQeViUU7a9k?e=RGHWvc

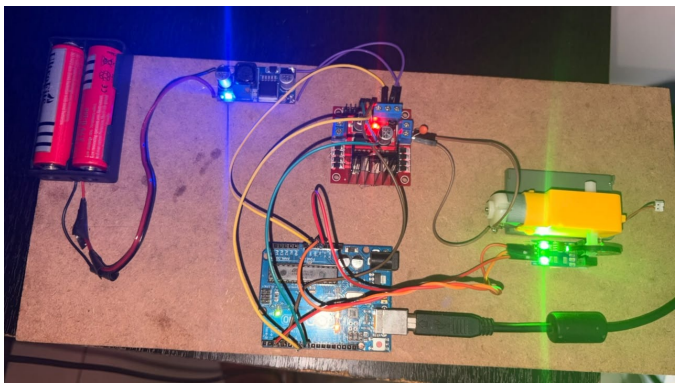


Fig. 8. Montaje físico: baterías 18650, convertidor LM2596, puente H L298N, Arduino Uno, motorreductor y sensor FC-03 con el disco ranurado.

VIII. Análisis de resultados

Los tres objetivos específicos se cumplieron: el modelo quedó simulado en tiempo real (Figs. 1 y 2), el firmware mide la velocidad y regula el PWM con un PID discreto, y la interfaz permite cambiar la consigna y las ganancias sin recompilar el programa.

La evidencia más clara del control es la Fig. 10: al aplicar un escalón de -35 rpm la velocidad sube, sobrepasa la consigna, presenta un segundo pico menor y se estabiliza con

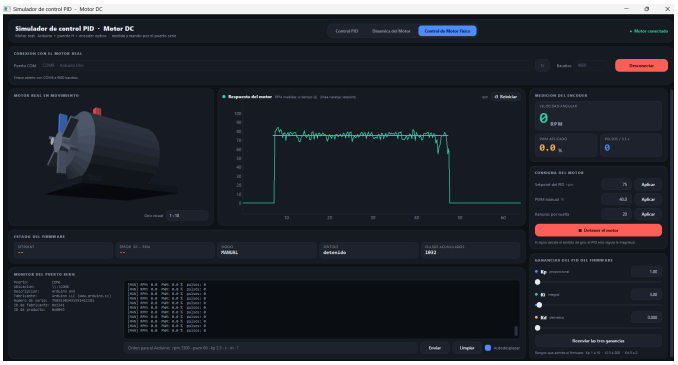


Fig. 9. Ensayo en lazo abierto con PWM fijo de 40 %: la velocidad ronda las 75 rpm con una dispersión apreciable por el ruido de medición.

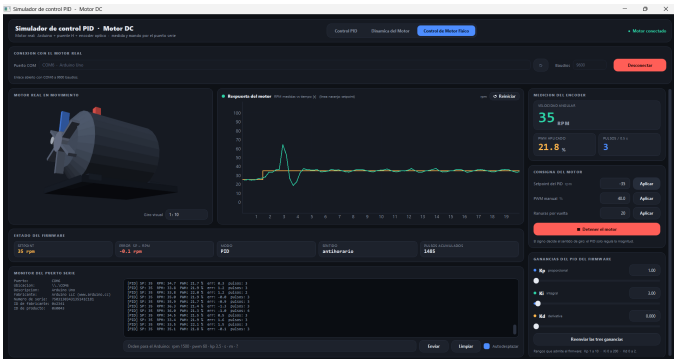


Fig. 10. Respuesta al escalón de -35 rpm con $K_p = 1$, $K_i = 3$ y $K_d = 0$: sobreimpulso, segundo pico menor y asentamiento sin error permanente.



Fig. 11. Ensayo largo con cambios de consigna: 35 rpm limpio, 100 rpm muy ruidoso y 75 rpm con un error de $-1{,}5$ rpm.

un error de apenas $-0{,}1$ rpm. Esa forma corresponde a una respuesta subamortiguada típica de un lazo PI, y la ausencia de error permanente confirma que la acción integral trabaja correctamente. La comparación con el ensayo en lazo abierto (Fig. 9) es igualmente reveladora: con PWM fijo la velocidad no se corrige, mientras que en lazo cerrado el controlador la lleva y la mantiene en el valor pedido.

El rango útil quedó entre unas 25 y 150 rpm. Por debajo, los pulsos llegan tan espaciados que el firmware interpreta el motor como detenido; por encima, el ruido de la conmutación del PWM produce cuentas de más, como se aprecia en el tramo de 100 rpm de la Fig. 11. El límite lo impone la medición y no el actuador: con PWM constante la lectura varía cerca de un ± 40 %, algo imposible para la inercia real de un motorreductor. De hecho, la velocidad máxima registrada, cercana a 600 rpm, triplica las 200 rpm que la hoja de datos del motorreductor TT de relación 1:48 especifica en vacío a 6 V (Adafruit Industries, s. f.), lo que confirma que a alta velocidad el encoder de 20 ranuras cuenta pulsos espurios. El rango confiable equivale, aproximadamente, al intervalo entre el 12 % y el 75 % de la velocidad nominal del motor.

IX. Conclusiones

El trabajo demuestra que es posible construir, con hardware de bajo costo, un lazo de control de velocidad funcional y, a la vez, una herramienta de simulación que ayude a entenderlo. Integrar ambas partes en una sola aplicación permitió anticipar el efecto de cada ganancia antes de ensayarla sobre el motor real.

Desde el punto de vista del control, tres decisiones marcaron la diferencia entre un lazo inestable y uno utilizable: normalizar el error respecto al fondo de escala real del motor, añadir una prealimentación que compense la zona muerta del conjunto motor–puente H, y proteger el integrador con anti-windup. Ninguna modifica la estructura del PID, pero todas resultan indispensables en un sistema físico con saturación y no linealidades.

La conclusión más importante es que la calidad del control nunca puede superar la calidad de la medición. La mayor limitación no fue el algoritmo ni la etapa de potencia, sino el encoder de 20 ranuras montado en el eje de salida, que degrada la lectura a medida que aumenta la velocidad. Mejorar la resolución del transductor, situándolo antes de la reductora o sustituyéndolo por un encoder de cuadratura, es la línea de trabajo futuro con mayor impacto sobre el desempeño global del sistema.

Bibliografía

Adafruit Industries. (s. f.). *DC gearbox motor — "TT motor" — 200RPM — 3 to 6VDC* [Hoja de datos]. Recuperado el 14 de septiembre de 2026.

Åström, K. J., y Hägglund, T. (1995). *PID controllers: Theory, design, and tuning* (2.ª ed.). Instrument Society of America.

Åström, K. J., y Hägglund, T. (2001). The future of PID control. *Control Engineering Practice, 9*(11), 1163–1175.

Dorf, R. C., y Bishop, R. H. (2011). *Sistemas de control moderno* (10.ª ed.). Pearson Educación.

Fatma, M. W., y Hamid, M. I. (2019). PWM speed control of DC permanent magnet motor using a PIC18F4550 microcontroller. *IOP Conference Series: Materials Science and Engineering, 602*, 012017.

Nise, N. S. (2011). *Control systems engineering* (6.ª ed.). John Wiley & Sons.

Ogata, K. (2010). *Ingeniería de control moderna* (5.ª ed.). Pearson Educación.

STMicroelectronics. (2000). *L298: Dual full-bridge driver* [Hoja de datos].